

■ Design Ad Click Aggregator

System Design Interview | Asynchronous Systems

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional

Count ad clicks per ad_id; Aggregate by time window (1min/1hr/1day); Query: clicks for ad X in last N minutes; Real-time and historical reporting

Non-Functional

Handle 1M+ clicks/sec; Query latency <1s; Tolerate duplicate clicks; At-least-once processing; Accurate billing (financial-grade)

Scale

Google Ads: trillions of clicks/year; clicks are high-volume, low-value events; aggregation reduces storage 1000×

■ High-Level Architecture

- Click Event → API Gateway (validates, deduplicates) → Kafka topic (partitioned by ad_id) → Stream Processor (Flink) → Aggregated DB
- Lambda Architecture: Speed layer (Flink real-time aggregates in Redis) + Batch layer (Spark hourly/daily on raw S3 data) → Serving layer merges
- Raw click events stored in S3 as source of truth; enables reprocessing if bugs found in aggregation logic
- Query service: reads from Redis (recent data) + columnar DB (ClickHouse/Druid) for historical; merges at query time

■ Click Deduplication

- **Bloom Filter:** Per ad_id bloom filter: if click_id seen before → drop; false positive rate ~0.1%; space efficient; reset hourly
- **Redis SET:** SADD dedup:{ad_id}:{minute} click_id → returns 0 if duplicate; expire key after 10 min window; accurate but memory-heavy
- **Idempotency Key:** click_id = hash(user_id + ad_id + timestamp_minute); same user clicking same ad in same minute = same key → deduplicated
- **Exactly-Once:** Flink checkpointing: state snapshot every 1 min; on failure restart from checkpoint; no double counting

■ Stream Aggregation (Apache Flink)

- Tumbling window: count clicks per ad_id per 1-minute bucket; emit count at window end → write to Redis + ClickHouse
- Sliding window: last 60 minutes rolling count per ad_id; more compute but enables real-time dashboards
- Watermarks: handle late-arriving events (network delay); events up to 30s late accepted; later events discarded or side-output
- State backend: RocksDB for large state (millions of ad_ids); checkpointed to S3 every 60s for fault tolerance

■ Storage Layer

- Real-time counts: Redis INCRBY ad_clicks:{ad_id}:{minute_bucket} — atomic, fast; TTL 2 hours; serves live dashboards
- Analytical DB: ClickHouse or Apache Druid — columnar storage, vectorized query engine; handles billions of rows; sub-second aggregation queries
- Raw events: S3 in Parquet format partitioned by date/hour; queried by Spark for batch reconciliation and backfill
- Reconciliation: daily Spark job recomputes exact counts from raw S3 data; compares with stream output; corrects billing DB if mismatch

■ Processing Architecture

Lambda Architecture	Kappa Architecture
✓ Batch layer for accuracy	✓ Single stream pipeline
✓ Speed layer for real-time	✓ Replay from Kafka for correction
✓ Two codebases to maintain	✓ Simpler to operate
✓ Strong consistency on batch	✓ Kafka retention cost

■ Billing Accuracy & Fraud

- Click fraud: ML model scores each click (IP reputation, user agent, click rate, conversion rate); flag suspicious clicks before aggregation
- Invalid clicks filtered before billing aggregation; kept in separate table for audit; advertisers see net valid clicks
- Billing pipeline separate from analytics: stronger exactly-once guarantees; financial audit trail; monthly reconciliation report
- Rate limiting per IP/user per ad: max 1 billable click per ad per user per hour; subsequent clicks tracked but not billed